	SR UNIVERSITY Campus Warangal
	Program: II - B. Tech (CS& AI) Batch:
	Name: N.KRUTHIKA Reg. No:2503A52584
	Department: Computer Science and AI Semester: I
	AI Assisted Coding-Assignment Date:21-09-2026

1.Question

2.Prompt

3.Code

4.Output

5.Observation

Q1 – AI-Assisted Bubble Sort Analysis

Tasks: Implement Bubble Sort in ascending order, count comparisons and swaps, explain one complete pass, test at least three input lists, ask about early-exit optimisation, and compare the original and improved versions.

PROMPT:

numbers=[64,34,25,12,22,11,90]

generate a python code implementing bubble sory and also explain why one complete pass of bubble sort is needed.implement the code to count comparisions and swaps ,testing atleast 3 input lists. implement the code to compare the original and improed versions.and also explain about an early exti optimization

CODE:

```python
def bubble_sort_original(numbers):
    """Bubble sort with the usual fixed passes (no early-exit check)."""
    result = numbers.copy()
    comparisons = swaps = 0

    for pass_number in range(len(result) - 1):
        for index in range(len(result) - 1 - pass_number):
            comparisons += 1
            if result[index] > result[index + 1]:
                result[index], result[index + 1] = result[index + 1], result[index]
                swaps += 1

    return result, comparisons, swaps


def bubble_sort_improved(numbers):
    """Bubble sort with early exit when a pass makes no swaps."""
    result = numbers.copy()
    comparisons = swaps = 0

    for pass_number in range(len(result) - 1):
        swapped = False
        for index in range(len(result) - 1 - pass_number):
            comparisons += 1
            if result[index] > result[index + 1]:
                result[index], result[index + 1] = result[index + 1], result[index]
                swaps += 1
                swapped = True

        # A complete pass without swaps means every adjacent pair is ordered.
        if not swapped:
            break
```

Acti
Go to

***OUTPUT*:**

```
Input: [64, 34, 25, 12, 22, 11, 90]
Original: [11, 12, 22, 25, 34, 64, 90] | comparisons=21, swaps=14
Improved: [11, 12, 22, 25, 34, 64, 90] | comparisons=21, swaps=14

Input: [1, 2, 3, 4, 5]
Original: [1, 2, 3, 4, 5] | comparisons=10, swaps=0
Improved: [1, 2, 3, 4, 5] | comparisons=4, swaps=0

Input: [5, 4, 3, 2, 1]
Original: [1, 2, 3, 4, 5] | comparisons=10, swaps=10
Improved: [1, 2, 3, 4, 5] | comparisons=10, swaps=10

Why complete passes are needed:
Each pass compares adjacent values and moves the largest unsorted value
to the end of the unsorted section. Repeating passes places all values;
a no-swap pass confirms the entire list is sorted.
Early-exit optimization: the improved version stops after that no-swap
pass, avoiding unnecessary comparisons on already sorted (or nearly sorted) lists.
```

OBSERVATION:

Input	Sorted Output	Comparisons	Swaps
[64,34,25,12,22,11,90]	[11,12,22,25,34,64,90]	21	14
[1,2,3,4,5]	[1,2,3,4,5]	10	0
[5,4,3,2,1]	[1,2,3,4,5]	10	10

Conclusion: Early-exit Bubble Sort avoids unnecessary passes when the list is already sorted or nearly sorted.

Both versions sort the list in ascending order. The improved version is more efficient for already sorted or nearly sorted lists because it stops when no swaps are needed. The number of comparisons and swaps depends on the input list.

Q2: Linear Search vs Binary Search

Tasks: Implement Linear Search and Binary Search, count comparisons, test first/middle/last/absent elements, explain why Binary Search requires sorted data, compare use cases, and suggest a valid optimisation.

PROMPT:

numbers=[5,12,18,23,31,45,56,72,89]

target=45

genrerte a python code for implementing linear search and binary search .implement the code to count the comparisions of both algorithms an test free,middle,last and absent elements, also implement the code to compare the results and explain the suitable cases.Explain why binary search requires sorted data and also suggest a valid optimisation

CODE:

```python
numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]


def linear_search(values, target):
    """Return (index, comparisons); works on sorted or unsorted input."""
    comparisons = 0
    for index, value in enumerate(values):
        comparisons += 1  # Compare the current value with the target.
        if value == target:
            return index, comparisons
    return -1, comparisons


def binary_search(values, target):
    """Return (index, comparisons); requires ascending-sorted input."""
    low, high = 0, len(values) - 1
    comparisons = 0
    while low <= high:
        middle = (low + high) // 2
        comparisons += 1
        if values[middle] == target:
            return middle, comparisons
        comparisons += 1
        if target < values[middle]:
            high = middle - 1
        else:
            low = middle + 1
    return -1, comparisons


# "First" (the prompt's "free"), middle, last, and absent elements.
```

OUTPUT:

```
Case     Target  Linear (index, comparisons)  Binary (index, comparisons)
------------------------------------------------------------------------
first    5       (0, 1)                       (0, 5)
middle   31      (4, 5)                       (4, 1)
last     89      (8, 9)                       (8, 7)
absent   50      (-1, 9)                      (-1, 6)

Comparison:
Linear search is useful for small or unsorted lists and one-off searches;
it takes O(n) time. Binary search is better for large, sorted lists, taking
O(log n) time. Counts include equality and ordering checks, not loop operations.
Binary search needs sorted data because it uses ordering to discard half the
remaining range; without order, the target could be in the discarded half.
Optimization: if the same data is searched repeatedly, build a set once and
use membership checks (average O(1)); this uses extra memory and loses indexes.
```

COMPARISION:

Target	Position	Linear Comparisons	Binary Comparisons
5	0	1	3
31	4	5	1
89	8	9	4
50	-1	9	4

Both algorithms find the target successfully. Binary Search is more efficient for this sorted list because it eliminates half of the remaining search range at each step. For an absent element, the number of comparisons depends on the search method and the list size.

Q3 – Compare Sorting Algorithms

Tasks: Implement Bubble Sort, Selection Sort and Insertion Sort using the same input; count comparisons and swaps/shifts; explain strengths and weaknesses; identify the best algorithm for nearly sorted data; and compare experimental results with theoretical complexity.

```
data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]
```

PROMPT:

data=[45,12,78,34,23,89,5,67,10,56]

generate a python code of implementing 3 sorting algorithms-bubble sort,selection sort,insertion sort using the same input for all algorithms.implement the code to count comparisions and swaps and also for comparing experimental results along with theoritical complexity.explain the strengths and waekness of them and also suggest which algorithm nearly suits for sorted data

CODE:

```
"""Compare three sorting algorithms using the same input list."""

data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]


def bubble_sort(values):
    items = values.copy()
    comparisons = swaps = 0
    for end in range(len(items) - 1, 0, -1):
        swapped = False
        for index in range(end):
            comparisons += 1
            if items[index] > items[index + 1]:
                items[index], items[index + 1] = items[index + 1], items[index]
                swaps += 1
                swapped = True
        if not swapped:  # Allows O(n) best-case time on already sorted data.
            break
    return items, comparisons, swaps


def selection_sort(values):
    items = values.copy()
    comparisons = swaps = 0
    for start in range(len(items) - 1):
        minimum = start
        for index in range(start + 1, len(items)):
            comparisons += 1
            if items[index] < items[minimum]:
                minimum = index
        if minimum != start:
```

OUTPUT:

```
Input: [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]
Algorithm       Sorted result                          Comparisons  Swaps/moves
--------------------------------------------------------------------------------
Bubble sort     [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]          44           24
Selection sort  [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]          45            6
Insertion sort  [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]          31           24

Theoretical complexity (n = number of items):
Bubble sort: time O(n) best; O(n^2) average/worst; auxiliary space O(1)
Selection sort: time O(n^2) best/average/worst; auxiliary space O(1)
Insertion sort: time O(n) best; O(n^2) average/worst; auxiliary space O(1)

Strengths and weaknesses:
- Bubble sort is simple and can stop early, but generally performs many comparisons and swaps.
- Selection sort uses at most n-1 swaps, but always makes O(n^2) comparisons and is not stable.
- Insertion sort is simple, stable, and effective for small inputs; it can require O(n^2) shifts on reverse-sorted data.

For nearly sorted data, insertion sort is usually the best choice: it needs only a few shifts and takes O(n) time when alr
eady sorted.
```

COMPARISION:

Algorithm	Best	Average	Worst
Bubble Sort	$O(n)$ with early exit	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$

Strengths/weaknesses: Bubble Sort is simple but usually slow; Selection Sort performs few swaps but still takes $O(n^2)$ comparisons; Insertion Sort is simple and performs especially well on nearly sorted data.

Best for nearly sorted data: Insertion Sort.

All three algorithms sort the same input list into ascending order. Insertion Sort is generally the best choice for nearly sorted data because it requires fewer shifts when only a few elements are out of place. The actual comparison and swap/shift counts depend on the implementation and input. All three have \(O(n^2)\) average-case time complexity.

Q4 – Optimising a Student Search and Sort Program

Tasks: Write functions for highest mark, sorting and name search; create a straightforward solution; identify unnecessary operations; improve it using a technique you understand; and compare time complexity.

```
students = [
    ("Arun", 82),
    ("Bala", 65),
    ("Chitra", 91),
    ("Divya", 74),
```

```
    ("Esha", 88)
]
```

PROMPT:

GENERATE A PYTHON CODE FOR OPTIMISING A STUDENT SEARCH AND SORT PROGRAM. CREATE A FUNCTION FOR HIGHEST MARK, SORTING AND NAME SEARCH . CREATE A STRAIGHT FORWARD SOLUTION FIRST AND IDENTIFY UNNECESSARY OPERATIONS. MAKE A MORE EFFICIENTT APPROCH AND IMPLEMENT ONLY SUGGESTIONS . COMPARE ORIGINAL AND IMPROVED TIME COMPLEXITY

CODE:

```python
students = [
    ("Arun", 82),
    ("Bala", 65),
    ("Chitra", 91),
    ("Divya", 74),
    ("Esha", 88),
]


# Straightforward implementation
def highest_mark_original(records):
    # Sorting everything to find the maximum does unnecessary work.
    sorted_records = sort_students_original(records)
    return sorted_records[-1] if sorted_records else None


def sort_students_original(records):
    # Bubble sort is straightforward, but makes O(n^2) comparisons.
    result = records.copy()
    for end in range(len(result) - 1, 0, -1):
        for index in range(end):
            if result[index][1] > result[index + 1][1]:
                result[index], result[index + 1] = result[index + 1], result[index]
    return result


def search_student_original(records, name):
    for student in records:
        if student[0] == name:
            return student
    return None
```

OUTPUT:

```
Original highest mark: ('Chitra', 91)
Original sorted list: [('Bala', 65), ('Divya', 74), ('Arun', 82), ('Esha', 88), ('Chitra', 91)]
Original name search: ('Chitra', 91)
Improved highest mark: ('Chitra', 91)
Improved sorted list: [('Bala', 65), ('Divya', 74), ('Arun', 82), ('Esha', 88), ('Chitra', 91)]
Improved name search: ('Chitra', 91)

Time complexity (n = number of students):
Highest mark: O(n^2) by sorting -> O(n) with max
Sorting:      O(n^2) bubble sort -> O(n log n) built-in sort
Name search:  O(n) linear search -> O(1) average per indexed lookup
Index setup:  O(n) time and O(n) space (once for repeated searches)
```

COMPARISION:

Operation	Simple Approach	Improved Approach
Highest mark	O(n)	O(n)
Sorting	O(n log n)	O(n log n)
Name search	O(n)	O(1) average
Extra space	O(1)	O(n)

The improved program can perform sorting and repeated name searches more efficiently. Finding the highest mark still takes \(O(n)\) time, while built-in sorting typically takes \(O(n \log n)\). A dictionary is useful when many name searches are required, but creating it also requires \(O(n)\) time and space. For a very small list, the difference in execution time may be negligible.

Q5 – Optimisation Challenge: Best Combination

This problem asks for a combination of items whose total price is as high as possible without exceeding the budget. The formulation is a 0/1 knapsack problem because each item can be selected at most once.

```
items = [
    ("Laptop", 60000),
    ("Phone", 30000),
    ("Tablet", 20000),
    ("Watch", 10000),
    ("Headphones", 5000)
]

budget = 50000
```

PROMPT:

DESIGN A SOLUTION TO MAXIMISE TOTAL VALUE WITHOUT EXCEEDING THE BUDGET USING PYTHON. MAKE A SIMPLE APPROCH AND EXPLIAN IT BY COMPARING BRUTE FORCE , GREEDY AND DYNAMIC

PROGRAMMING. DETERMINE which approach guarantees an optimal solution for this formulation.IMPLEMENT THE SELECTED SOLUTION AND TEST AT LEAST THREE BUDGETS

CODE:

```python
from itertools import combinations


# Each amount is treated as both the item's cost and its value. Each item can
# be selected at most once (the 0/1 knapsack formulation).
items = [
    ("Laptop", 60000),
    ("Phone", 30000),
    ("Tablet", 20000),
    ("Watch", 10000),
    ("Headphones", 5000),
]


def brute_force(items, budget):
    """Check every subset; simple, but takes O(2^n) time."""
    best = ([], 0)
    for size in range(len(items) + 1):
        for group in combinations(items, size):
            total = sum(value for _, value in group)
            if total <= budget and total > best[1]:
                best = (list(group), total)
    return best


def greedy(items, budget):
    """Pick the highest-value affordable item repeatedly; not generally optimal."""
    chosen = []
    total = 0
    for item in sorted(items, key=lambda entry: entry[1], reverse=True):
        if total + item[1] <= budget:
            chosen.append(item)
```

OUTPUT:

```
Budget: 50,000
Selected: Phone, Tablet
Total value: 50,000

Budget: 45,000
Selected: Phone, Watch, Headphones
Total value: 45,000

Budget: 25,000
Selected: Tablet, Headphones
Total value: 25,000
```

COMPARISION:

Budget	One Optimal Combination	Maximum Total
₹25,000	Tablet + Headphones	₹25,000
₹50,000	Phone + Tablet	₹50,000
₹45,000	Phone + Watch + Headphones	₹45,000

Dynamic Programming is a suitable choice because each item can be selected at most once, and the total price must not exceed the budget. Brute Force also guarantees the optimum if it checks every combination, while Greedy does not guarantee the optimum for this problem.

For the given items and a budget of ₹50,000, the optimal combination is:

Phone — ₹30,000

Tablet — ₹20,000

Total price: ₹50,000

This combination uses the entire budget without exceeding it. The implementation should also be tested with at least two other budgets to verify that it finds the best valid combination in each case.

Final conclusion:

All five questions demonstrate how comparing an original approach with an improved one helps us understand algorithm efficiency. Counting comparisons, swaps or shifts, and analysing time and space complexity helps identify which algorithm is more suitable for different inputs. An improvement should be tested to confirm that it produces the correct output as well as better performance.